

Building and Securing a REST API

MoMo SMS Transactions API

Team dema3 | Chol Deng

Repository: <https://github.com/Chol1000/dema3>

1. Introduction to API Security

An API is the door through which every client application reaches the data behind it. For a mobile money system that data is highly sensitive: transaction amounts, account balances, phone numbers and the names of both parties to a payment. If that door is left unlocked, anyone who can reach the server can read every customer's financial history, or worse, modify and delete records. API security is therefore not an optional extra bolted on at the end; it is a basic requirement of the design.

Securing an API rests on three distinct ideas that are easily confused. **Authentication** answers "who is making this request?". **Authorization** answers "is this caller allowed to do this particular thing?". **Confidentiality** answers "can anyone else read this exchange while it travels across the network?", and is provided by transport encryption (HTTPS/TLS). A system needs all three. Strong authentication is worthless if the credential is transmitted in clear text, and encryption is worthless if every authenticated caller is allowed to delete anything.

This project implements the first of those three using HTTP Basic Authentication, as required by the assignment. Every endpoint rejects unauthenticated requests with a **401 Unauthorized** response. Section 5 examines honestly where that scheme falls short and what a production deployment would use instead.

2. System Overview and Data Parsing

The source dataset, `modified_sms_v2.xml`, is an SMS backup export containing 1,691 `<sms>` elements from an MTN Mobile Money account. The useful content sits in the body attribute of each element as free-form human-readable text, so the parser has to read meaning out of prose rather than out of structured fields.

`dsa/parse_xml.py` handles this with a list of labelled regular expressions. Each SMS body is tested against the patterns in order; the first match determines the transaction category and supplies the named capture groups for amount, fee, new balance, sender, receiver, account and transaction id. Amounts written as "12,500" are normalised to floats, and timestamps are converted to ISO-8601. Where a message carries no readable timestamp, the parser falls back to the epoch date attribute on the element.

The 1,691 records break down as follows:

Transaction type	Count	Transaction type	Count
payment_to_code_holder	658	otp	8
transfer_to_mobile	585	bank_transfer	6
bank_deposit	249	failed_transaction	5
incoming_money	63	withdrawal	3
bill_or_airtime_payment	53	reversal	2

Transaction type	Count	Transaction type	Count
third_party_payment	36	merchant_payment	2
bundle_purchase_notice	21		

Table 1 — Records per transaction type. All 1,691 records are classified; none fall into an unrecognised category.

The parser writes the result to `data/transactions.json` as a list of dictionaries, which is exactly the shape the API then serves.

3. API Endpoint Documentation

The API is written in plain Python on top of `http.server`, with no third-party dependencies. It exposes five CRUD endpoints over the parsed transactions. Records are held in memory in a dictionary keyed by `id`, so lookups by `id` are $O(1)$ (see Section 6). **Every endpoint requires Basic Authentication.**

Method	Endpoint	Purpose	Success
GET	/transactions	List all transactions	200
GET	/transactions/{id}	Retrieve one transaction	200
POST	/transactions	Create a transaction	201
PUT	/transactions/{id}	Update an existing record	200
DELETE	/transactions/{id}	Delete a record	200

Table 2 — CRUD endpoints.

3.1 Request and response examples

GET /transactions/1 — retrieve a single record

```
$ curl -u admin:momo2025 http://localhost:8000/transactions/1
HTTP/1.0 200 OK
Content-Type: application/json

{
  "id": 1,
  "transaction_type": "incoming_money",
  "amount": 2000.0,
  "new_balance": 2000.0,
  "sender": "Jane Smith",
  "sender_phone": "*****013",
  "financial_transaction_id": "76662021700",
  "timestamp": "2024-05-10T16:30:51"
}
```

POST /transactions — create a record. `transaction_type` and `amount` are required; the server assigns the `id`.

```
$ curl -u admin:momo2025 -X POST http://localhost:8000/transactions \
  -H "Content-Type: application/json" \
  -d '{"transaction_type":"incoming_money","amount":7500,"sender":"Chol Deng"}'
HTTP/1.0 201 Created

{
  "id": 1692,
  "transaction_type": "incoming_money",
  "amount": 7500.0,
  "sender": "Chol Deng"
}
```

PUT /transactions/1692 — partial update; send only changed fields.

```
$ curl -u admin:momo2025 -X PUT http://localhost:8000/transactions/1692 \
  -H "Content-Type: application/json" -d '{"amount":9900}'
HTTP/1.0 200 OK
```

DELETE /transactions/1692 — remove a record.

```
$ curl -u admin:momo2025 -X DELETE http://localhost:8000/transactions/1692

HTTP/1.0 200 OK
{ "message": "Transaction 1692 deleted", "deleted": { ... } }
```

Unauthorised request — wrong password.

```
$ curl -i -u admin:wrongpassword http://localhost:8000/transactions

HTTP/1.0 401 Unauthorized
WWW-Authenticate: Basic realm="MoMo Transactions API"

{ "error": "Unauthorized: invalid username or password", "status": 401 }
```

3.2 Error codes

Code	Name	When it occurs
200	OK	Successful GET, PUT or DELETE
201	Created	Successful POST
400	Bad Request	Malformed JSON, missing required field, unknown field, or a negative / non-numeric amount
401	Unauthorized	Absent, malformed or incorrect Basic Auth credentials
404	Not Found	Unknown transaction id, or unknown endpoint

Table 3 — Error codes. Every error shares the shape `{ "error": ..., "status": ... }`. Full documentation is in `docs/api_docs.md`.

4. Authentication Implementation

Each request must carry an `Authorization` header holding the Base64 encoding of `username:password`. The handler decodes it and compares both halves against the configured credentials before any route logic runs, so no endpoint can be reached without passing the check.

```
Authorization: Basic YWRtaW46bW9tbzIwMjU=
```

Three implementation details are worth highlighting:

- **Constant-time comparison.** The credentials are checked with `hmac.compare_digest()` rather than `==`. A normal string comparison stops at the first differing character, and an attacker able to measure that timing difference could recover the password one character at a time. `compare_digest()` always takes the same time, closing that side channel.
- **Credentials are not hardcoded.** They are read from the `API_USERNAME` and `API_PASSWORD` environment variables, with development defaults, so the deployed secret never needs to live in source control.
- **Every failure mode returns 401.** A missing header, a header that is not Basic, corrupt Base64, and a simply wrong password are all handled and all answered with 401 plus a `WWW-Authenticate` challenge — never a 500.

5. Reflection: Why Basic Auth Is Weak

Basic Authentication is implemented correctly here, but the scheme itself is weak, and for real mobile money data it would not be acceptable.

5.1 Base64 is encoding, not encryption

This is the central flaw. Base64 uses no key and no secret, so the transformation is trivially reversible by anybody:

```
$ echo -n 'admin:momo2025' | base64
YWRtaW46bW9tbzIwMjU=

$ echo 'YWRtaW46bW9tbzIwMjU=' | base64 --decode
admin:momo2025
```

The password is therefore effectively sent in plain text. Over plain HTTP anyone able to observe the traffic — someone sharing a public Wi-Fi network, a compromised router, an ISP — reads the real password directly off the wire. Basic Auth is only ever tolerable over HTTPS, and even then the problems below remain.

5.2 Further limitations

- **The password is replayed on every request.** Because HTTP is stateless, the credential is resent with each call, multiplying the chances of it leaking into a proxy log, a crash report or a screenshot. One leak exposes it permanently.
- **No expiry and no revocation.** The credential stays valid until someone manually changes the password, and changing it breaks every other client using that same account.
- **A single shared account.** With one `admin` credential there is no audit trail — the log shows that "admin" deleted a record, not who — and no least privilege, since a read-only mobile client holds a credential that can also DELETE.
- **No brute-force protection.** The implementation has no rate limiting, lockout or second factor, so an attacker can guess passwords as fast as the server responds.

5.3 Stronger alternatives

JWT (JSON Web Tokens). The client authenticates once at a login endpoint and receives a signed token, sent afterwards as `Authorization: Bearer <token>`. The token carries claims such as user id, role and an expiry timestamp, and is signed with a server-held key. This means the real password crosses the network only once, tokens expire on their own so a stolen one is useful only briefly, and the embedded claims enable per-user audit logs and role-based access. The trade-off is that a JWT is signed but not encrypted, so its payload is still readable and must never hold secrets; and because verification is stateless, tokens are kept short-lived and backed by a refresh flow rather than revoked directly.

OAuth 2.0. Where third-party applications are involved, an authorization server issues scoped access tokens so the third party never sees the user's password at all. For a MoMo system a budgeting app could be granted `transactions:read` and nothing more — able to list transactions but never delete one — and the user could revoke that single app at any time without changing their own password. The cost is significantly greater complexity and the need to run an authorization server, which is justified for multi-client access but excessive for one trusted internal client.

Supporting measures. Whichever scheme is chosen, HTTPS/TLS is mandatory, passwords should be stored salted and hashed with bcrypt or Argon2, login endpoints should be rate limited, and per-user accounts with roles should replace the single shared login so that actions are attributable.

6. Data Structures and Algorithms Comparison

Two strategies for finding a transaction by id were implemented in `dsa/dsa_comparison.py` and timed against each other:

- **Linear search** — walk the list of transactions one record at a time, comparing each id until the target is found.
- **Dictionary lookup** — build a dictionary mapping id to transaction once, then fetch the target by key.

Both were run against all 1,691 records for **20 target ids** spaced evenly across the dataset, with **1,000 repetitions per measurement** using Python's `timeit` module. Each result was asserted equal across the two methods so the comparison is like for like.

Target id	Position in list	Linear search (µs)	Dictionary (µs)	Speed-up
1	1	0.082	0.065	1.3x
85	85	1.865	0.065	28.8x
169	169	3.665	0.064	57.0x
253	253	5.741	0.065	88.5x
337	337	8.003	0.065	123.9x
421	421	10.153	0.065	156.6x
505	505	12.116	0.064	189.8x
589	589	14.191	0.066	216.0x
673	673	16.121	0.063	254.9x
757	757	18.248	0.065	280.2x
841	841	20.365	0.065	313.7x
925	925	22.371	0.065	345.5x
1009	1009	24.495	0.065	379.3x
1093	1093	26.518	0.065	410.6x
1177	1177	28.395	0.065	436.6x
1261	1261	30.565	0.063	481.7x
1345	1345	32.018	0.066	485.4x
1429	1429	34.878	0.065	533.8x
1513	1513	36.650	0.065	561.3x
1597	1597	38.320	0.065	591.8x
AVERAGE		19.238	0.065	297.1x

Table 4 — Measured lookup cost. Full output is captured in `screenshots/dsa_results.txt`.

6.1 Why dictionary lookup is faster

The table shows the pattern clearly. Linear search costs rise steadily with how deep the record sits in the list — roughly 0.082 µs for the first record against 38.320 µs for one near the end — because the number of comparisons grows in direct proportion to the position. That is **O(n)** behaviour. Dictionary lookup, by contrast, stays flat at about 0.065 µs no matter where the record sits, which is **O(1)**.

The reason is that a dictionary does not search at all. Python hashes the key once, and that hash tells it directly which bucket the value lives in, so the work done is the same whether the dictionary holds ten records or ten thousand. Linear search has no such shortcut and must compare records one by one until it finds a match. Averaged across the 20 sampled ids, the dictionary is about **297.1x faster** on this dataset.

The trade-off is memory and setup: the dictionary has to be built first, an $O(n)$ pass, and it holds an extra reference per record. That cost is paid once and repaid on every subsequent lookup, which is why the API itself stores transactions in a dictionary keyed by id — making every GET, PUT and DELETE by id an $O(1)$ operation.

6.2 Other structures that would improve on linear search

- **Binary search** on an id-sorted list — $O(\log n)$, halving the search space each step. It needs no extra memory beyond keeping the list sorted, but every insertion must preserve that order.
- **B-tree or database index** — how a real database indexes a primary key. It keeps lookups fast while also supporting range queries a hash cannot answer, such as "all transactions between two dates".
- **Hash index on secondary fields** — building the same id-to-record trick for sender or `transaction_type` would extend $O(1)$ lookup beyond the primary key.

7. Testing and Validation

The API was tested end to end with `curl` via `screenshots/run_tests.sh`, which exercises nine cases and captures the full request/response transcript to `screenshots/api_test_results.txt`.

#	Test case	Expected
1	GET /transactions with valid credentials	200
2	GET /transactions/1 with valid credentials	200
3	GET /transactions with wrong password	401
4	GET /transactions with no credentials	401
5	POST /transactions creating a record	201
6	PUT on the newly created record	200
7	DELETE the newly created record	200
8	GET the deleted record	404
9	POST with a non-numeric amount	400

Table 5 — Test cases. All nine produced the expected status code.

The script reads the server-assigned id back from the POST response rather than assuming one, so the whole suite can be re-run repeatedly against a running server without the update, delete and 404 cases drifting out of step.

8. Conclusion

The system parses all 1,691 SMS records into structured JSON, serves them through five CRUD endpoints built on the Python standard library, and protects every endpoint with Basic Authentication that correctly returns 401 for missing, malformed and incorrect credentials. The DSA comparison demonstrates in measured numbers why the API stores its records in a dictionary: an average 297.1x

speed-up over linear search, and a lookup cost that stays flat as the dataset grows.

The main limitation is the authentication scheme itself. Because Base64 is encoding rather than encryption, the password is effectively sent in the clear on every single request, with no expiry, no revocation and no per-user identity. The next step for this project would be to serve the API over HTTPS and replace Basic Auth with short-lived JWTs, adding OAuth 2.0 scopes once third-party applications need access to the data.